

Getting Started

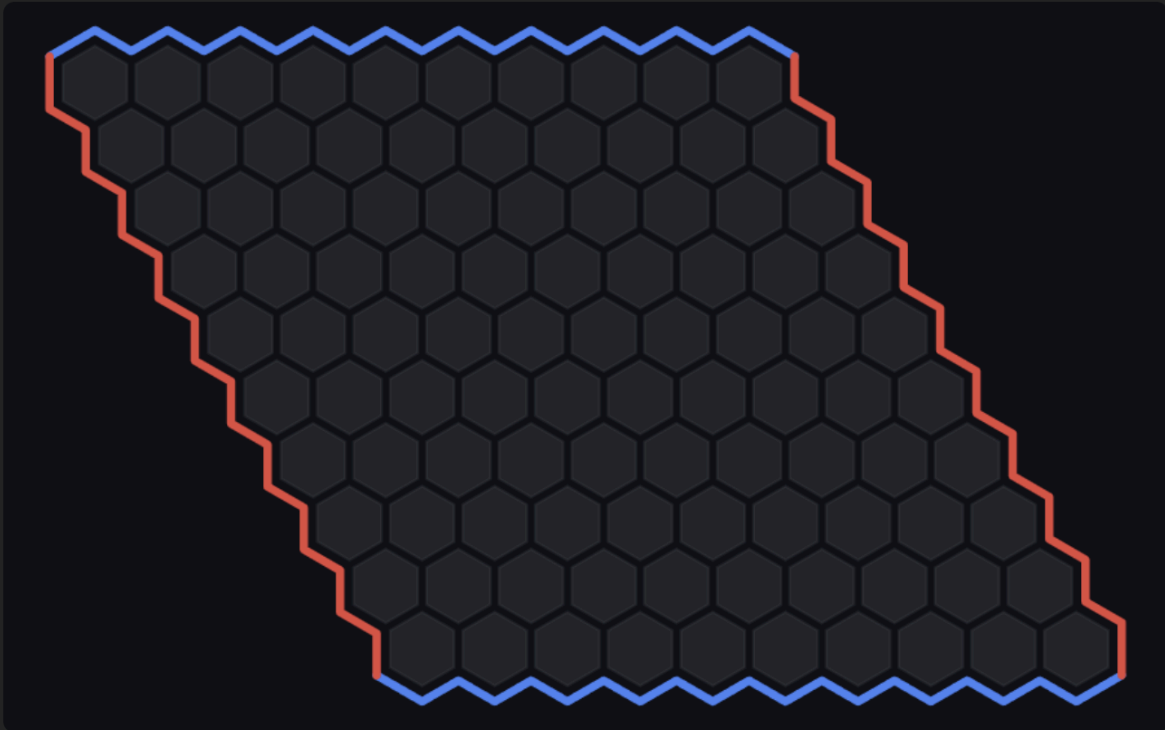
To get started, first familiarize yourself with the game.

This game we will be playing is named **Hex**.

After learning about the game, we will guide you through setting up your local development environment and the development process.

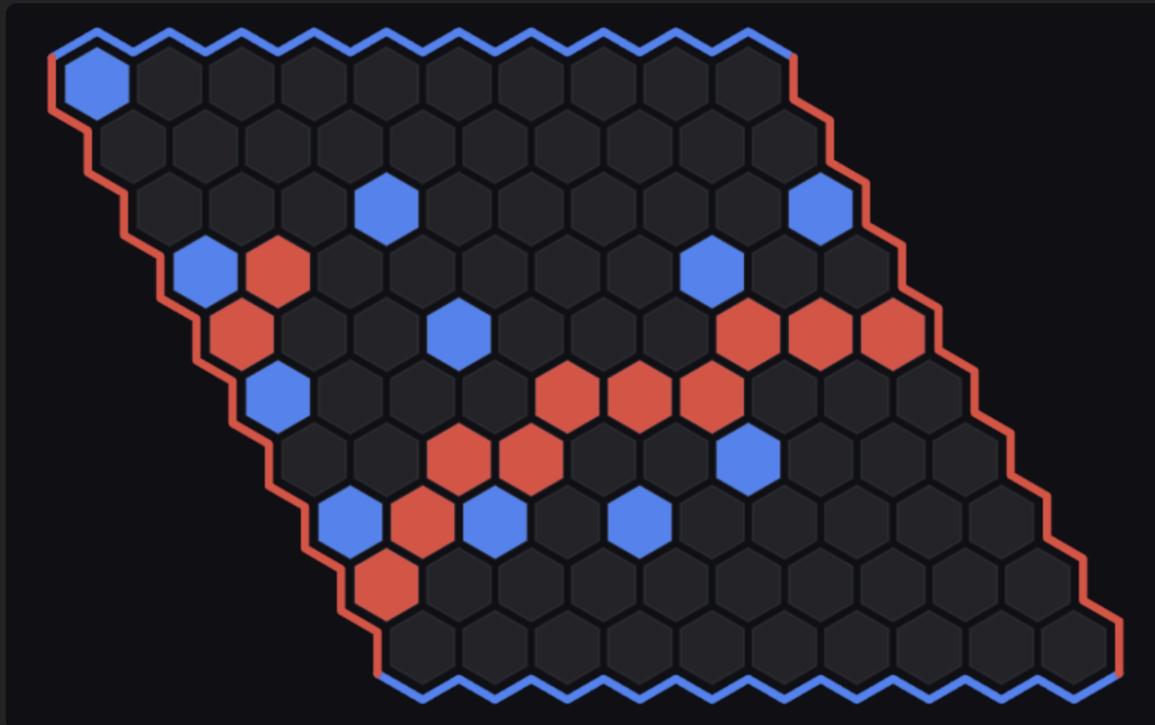
Hex

Hex is a strategic, two-player game where both players take turns placing tiles of their color onto a board comprised of hexagonal spaces:



Each player has a color, red or blue.

The goal is to form a path of tiles of your own color to connect the two sides of your color:



Both players place tiles anywhere on the board in each turn until this goal is reached.

The first player to reach their goal wins the game.

Hex is a simple game yet can lead to complex strategies making it an interesting game for a competition like this one.

You can find an interactive Hex tutorial [here](#) also including online play and extra resources.

i Important: we currently do not use or implement the swap rule.

Environment

You will need a laptop with python and an IDE of your choice installed.

We recommend python 3.12 and VS Code.

A package manager like [uv](#) or [conda](#) is also recommended.

The main requirement to get started is a python environment with the [gamelib](#) package installed.

You can install it using pip (don't forget to activate your python environment if applicable):

```
pip install aica-gamelib
```

You may also use the [numpy](#) package if needed (not recommended for your first implementation).

If you do use it, your code must be compatible with numpy version 2.4.2.

i Other packages outside of the python standard library are not allowed!

For more information on the [gamelib](#) package and how to work with it check its dedicated [documentation page](#).

Development

These are our recommended first steps (after setting up the environment and checking out the other documentation):

1. Copy a sample agent file into your local environment and check that it is working
2. Make some small changes to play around and understand how it works.
3. Think of a strategy that you want to try
4. Implement it and try playing against it yourself
5. Upload it to the Game AI Platform and see how it performs against other competitors

Gamelib

Gamelib is our custom python package for the Game AI Platform.

It contains interfaces, library code and examples for all supported games.

Its PyPi page can be found [here](#).

Creating an Agent

To create a game AI agent, you need to create a python file fulfilling the following conditions.

An example for such a file can be found at the end of this documentation page.

Your agent file must end be named "agent.py" or end with "_agent.py"

An example for a valid filename is: `test_agent.py`

The script must containing a class that inherits from `gamelib.hex.agent.Agent`.

You must implement two methods:

```
initialize(self, agent_init_data)
```

```
get_move(self, game_state)
```

```
Move
```

All agents must have the following code block at the end of the script.

Otherwise the agent will not run properly on the Game AI Platform.

```
# the rest of your file if __name__ == "__main__": agent = MyHexAgent() #  
rename this to your agent class agent.start()
```

This is not for local use but is run by the platform.

To run your agent locally use the `gamelib-play` console command while in your environment:

```
// play against your agent manually gamelib-play hex test_agent.py human
// let two of your local agents play against eachother gamelib-play hex
test1_agent.py test2_agent.py
```

Run `gamelib-play --help` for more info.

Important: When running on the platform there will be a time limit of 5 seconds for the `get_move` function to return a move. You are responsible for ensuring this limit is upheld.

Game Concepts

GameState

The `GameState` object passed to `get_move` is defined in `gamelib.hex.gamestate`.

It contains all data on the current state of the game.

It has the following attributes:

```
board_size: int
board: list[list[int]]
turn: int
status: int
```

The size of our board will be 11, giving us a board with 11×11 spaces for tiles.

In the board attribute of the state object, `-1` represents empty while `0` and `1` represent an existing tile from either player 0 or 1.

Agents receive their player id (0 or 1) from the `init_data` object's `player_id` key (see example below).

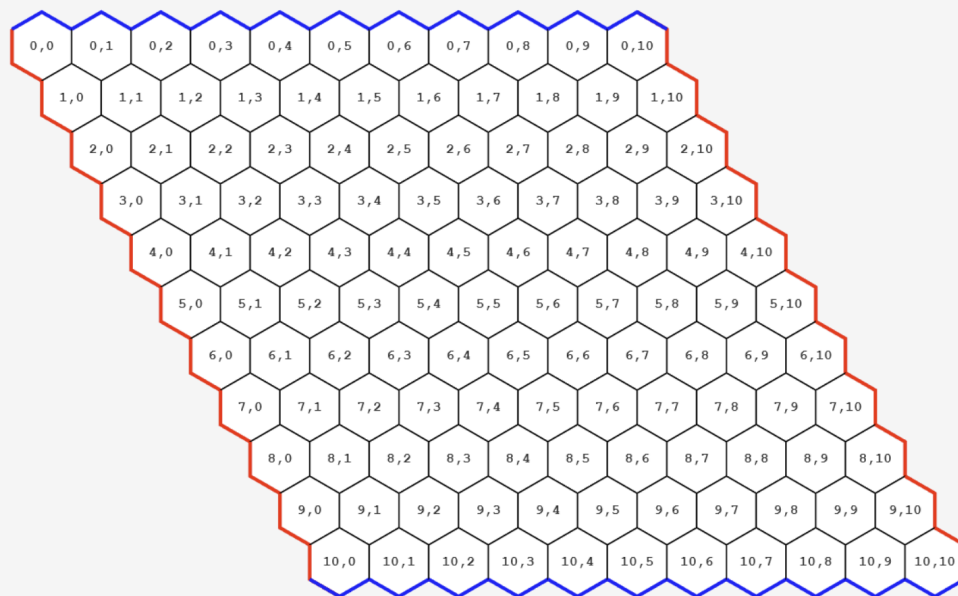
`turn` represents who's turn it currently is (who will place the next tile).

Thus it is either 0 or 1 for each player id.

`status` represents the current status of the game and contains a value from the following Enum:

```
class GameState(Enum): ONGOING = -1 DRAW = -2 PLAYER_0_WINS = 0  
PLAYER_1_WINS = 1
```

Board indices look like this:



Move

The `Move` object returned by `get_move` is defined in `gamelib.hex.move`.

It has two attributes:

- `player: int`
- `position: list[int]`

`player` contains the id of the player making the move (0 or 1).

`position` must be a list of 2 integers containing the row and column indices (in that order) of the tile to be placed. Example: `[1, 2]` designates the space in the second row and third column (indices start at zero, see image above).

Engine

The engine object is used internally to validate and apply moves to game states.

You may also import and use its methods in your code if needed:

```
from gamelib.hex.engine import Engine
from gamelib.hex.move import Move
from gamelib.hex.gamestate import GameState as State

engine = Engine()
state = State.initial() # initialize an empty board
move = Move(player=0, position=[2,3])
valid = engine.validate_move(state, move) # check if move is valid
if valid:
    new_state = engine.apply_move(state, move) # apply move to state
```

Example

Here is a simple example of a hex agent that selects a random available cell on the board.

```
import os
import random
from typing import override

from gamelib.hex.agent import Agent
from gamelib.hex.gamestate import GameState as State
from gamelib.hex.move import Move

class HexAgent(Agent):
    """
    A simple Hex agent that picks a random available cell.
    """

    @override
    def initialize(self, init_data: dict) -> None:
        """Initialize the agent."""
        self.player_id = init_data["player_id"]

    @override
    def get_move(self, state: State) -> Move:
        """Find a random empty cell and return as move."""
        empty_cells = [(r, c) for r in range(state.board_size) for c in range(state.board_size) if
state.board[r][c] == -1]
        if not empty_cells:
            raise ValueError("No valid moves available.")

        position = random.choice(empty_cells)
        return Move(player=self.player_id, position=list(position))

if __name__ == "__main__":
    # Instantiate and start the agent (run by the platform)
```

```
# Keep this and don't remove if submitting to Game AI Platform.  
agent = HexAgent()  
agent.start()
```



Possible Approaches

General Hex Strategy

As mentioned previously, the goal is to make a connection between your two sides by placing tiles of your color.

Of course, your opponent is doing the same.

An interesting note here is that ties are not possible in Hex.

There is always a winner when the board is full of tiles.

This means obstructing your opponent is actually a valid strategy for winning the game.

A comprehensive resource on Hex strategy can be found [here](#) (note: we don't use the swap rule).

Possible Approaches

As an additional help, we have thought of some possible approaches to get started with your strategy.

This is by no means a complete list and these approaches must be extended and combined to achieve an actually good solution. They are meant as an inspiration.

Board Eval

One of the first things we recommend implementing is a board evaluation function.

It should return either the best possible move or a score for each possible move.

Evaluation can be done for example by searching for shortest paths between the two sides or for obstruction possibilities.

The board can be modeled as a graph and traversed as such.

Additional manual heuristics and calculations may also be used.

An important notion here is that certain patterns of tiles can not be obstructed (prevented from being connected by the enemy) although they are not touching.

Tree Search

Tree search through the moves and game states is an important strategy for all good Hex agents.

Different traversal strategies are possible:

Minimax

Minimax is an algorithm that explores possible future moves by assuming the AI tries to maximize its score while the opponent tries to minimize it, then chooses the move with the best guaranteed outcome. A scoring function allows the algorithm to compare and rank positions.

Monte Carlo

Monte Carlo Tree Search (MCTS) is an algorithm that evaluates moves by running many simulated playthroughs from each possible position and choosing the move that performs best on average. A simple scoring function could assign 1 for a win and 0 for a loss, with each move scored by its average result across all simulations.

Game AI Platform

This documentation covers the Game AI Platform.

The platform is a custom-built framework for competing in turn-based games with coded AI agents.

It's website can be found [here](#).

The first time you use our platform, you will have to create an account.

Remember to verify your email.

After doing so, you may sign into the platform.

You may use the platform to test your locally developed agent against your competitors.

The following assumes you already have a completed agent file.

In the Game AI Platform, every user has a certain number of agents per game (ex.: 1 for Hex).

Each agent contains some code that defines how it plays the game.

You can change this logic by changing the uploaded code (the submission).

Agents play against each other with their current game logic in matches that can be viewed on the platform.

In the end of the hackathon there will be a tournament on the platform.



You do not have to submit your code to the platform until the tournament.

Submitting code early and letting it play against other competitors is completely optional.

Creating your first Agent

1. Create a zip file containing your agent script
2. Sign into the platform
3. Navigate to the “Games” tab in the sidebar
4. Click on “Hex”
5. Click on “Create New Agent”
6. Name your agent
7. Click “Browse Files” and select the zip containing your agent
8. Optionally: name your submission (the code you uploaded)
9. Click “Create Agent”
10. Wait for your submission to be built and the agent to be created
11. Check that the submission is active

✓ Make sure your agent file follows the defined naming scheme and other conditions.
Make sure the zip directly contains the agent file and no unnecessary files and directories.

i If the submission fails, the agent will still be created.
You can then check the submission build logs and fix the error and upload a new submission.

You can set the active submission for your agent at any time.

Simply, click “Use for Agent” next to the submission in the agent screen.

The next match the agent participates in will use its updated code.

Explore the Platform

The match scheduler will continuously schedule new matches between existing agents with valid submissions.

You can view past and live matches in the platform [here](#).

Also have a look at the [leaderboard](#) to see how your agent is doing in comparison to the others.

The dashboard can help you get a quick overview over your current work.

Later, you can visit the tournament tab to watch the tournament progress in real time!